ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 9

Refresh-токены

Рассмотрим процесс работы с refresh-токенами. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Разделение access- и refresh-токенов

В классических веб-приложениях токены играют следующие роли: access-токен является токеном доступа, то есть его передача в заголовке является некоторой "подписью", валидируя которую, сервер понимает, что запрос был отправлен конкретным пользователем, в последствии чего сервер может оценить его уровень прав в системе и ответить на запрос надлежащим образом.

Токен доступа, он же access-токен, имеет ряд параметров, среди которых зачастую встречается `expiresIn` - дата, до которой токен является валидным, обычно он устанавливается на небольшое значение (например, через 15 минут после выпуска). Параметр существует для того, чтобы выпущенный токен имел не слишком долгий жизненный цикл и потенциальным злоумышленникам было труднее получить доступ к системе, получив токен тем или иным образом.

Если токен доступа быстро истекает, возникает проблема, так как пользователю приходится выполнять вход в аккаунт (вводить логин и пароль) каждый раз по истечению пятнадцати минут, что очень сильно снижает удобство и пользовательский опыт.

На помощь приходят токены обновления - они же refresh-токены. Такой токен обычно идёт в паре с access-токенов (выпускается одновременно с ним), но имеет значительно более долгий цикл жизни (например, неделю или месяц). Механизм заключается в том, что на клиенте хранятся сразу два этих токена, и когда истекает токен доступа и от сервера приходит соответствующая ошибка, клиент пытается в

автоматическом режиме отправить дополнительный запрос на энд-поинт, который отвечает за выпуск новой пары access- + refresh-токенов с целью получить новый токен доступа, после чего повторяет изначальный запрос с новым, актуальным access-токеном. Таким образом, если пользователь осуществляет взаимодействия с системой постоянно, вводить свои данные заново ему не приходится.

Бывают и другие модификации такого механизма. Например такие, при которых энд-поинт генерации токена доступа по токену обновления не генерирует новый refresh-токен, а оставляет старый, что означает, что при истечении срока действия refresh-токена, пользователю в любом случае придётся ввести учётные данные (пароль), то есть он будет вводить их каждую неделю, месяц, или любой другой период, на который генерируется токен обновления.

Генерация refresh-токенов

Генерация refresh-токена выглядит точно так же, как и генерация access-токена, но с другими параметрами:

```
jwt.sign(
    {
        sub: user.id,
        username: user.username,
    },
    REFRESH_SECRET,
    {
        expiresIn: REFRESH_EXPIRES_IN,
    }
);
```

Где REFRESH_SECRET - отдельный секрет для токенов обновления, а REFRESH_EXPIRES_IN - время истечения токена, например "7d".

Пример приложения

Модифицируем приложение из прошлого практического занятия, добавив генерацию токенов обновления.

Предлагается вынести генерацию токенов в отдельные функции `generateAccessToken()` и `generateRefreshToken()`

Другая отличительная особенность будет заключаться в том, что мы создадим локальное хранилище с refresh-токенами и будем удалять неактуальные.

```
const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
```

```
const app = express();
app.use(express.json());

const PORT = 3000;

// Секреты подписи
const ACCESS_SECRET = "access_secret";
const REFRESH_SECRET = "refresh_secret";

// Время жизни токенов
const ACCESS_EXPIRES_IN = "15m";
const REFRESH_EXPIRES_IN = "7d";

// { id, username, passwordHash }
const users = [];

// Хранилище refresh-токенов в памяти
const refreshTokens = new Set();

function generateAccessToken(user) {
  return jwt.sign(
    {
      sub: user.id,
      username: user.username,
    },
    ACCESS_SECRET,
    {
      expiresIn: ACCESS_EXPIRES_IN,
    }
  );
}

function generateRefreshToken(user) {
  return jwt.sign(
    {
      sub: user.id,
      username: user.username,
    },
    REFRESH_SECRET,
    {
      expiresIn: REFRESH_EXPIRES_IN,
    }
  );
}

app.post("/api/auth/register", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
```

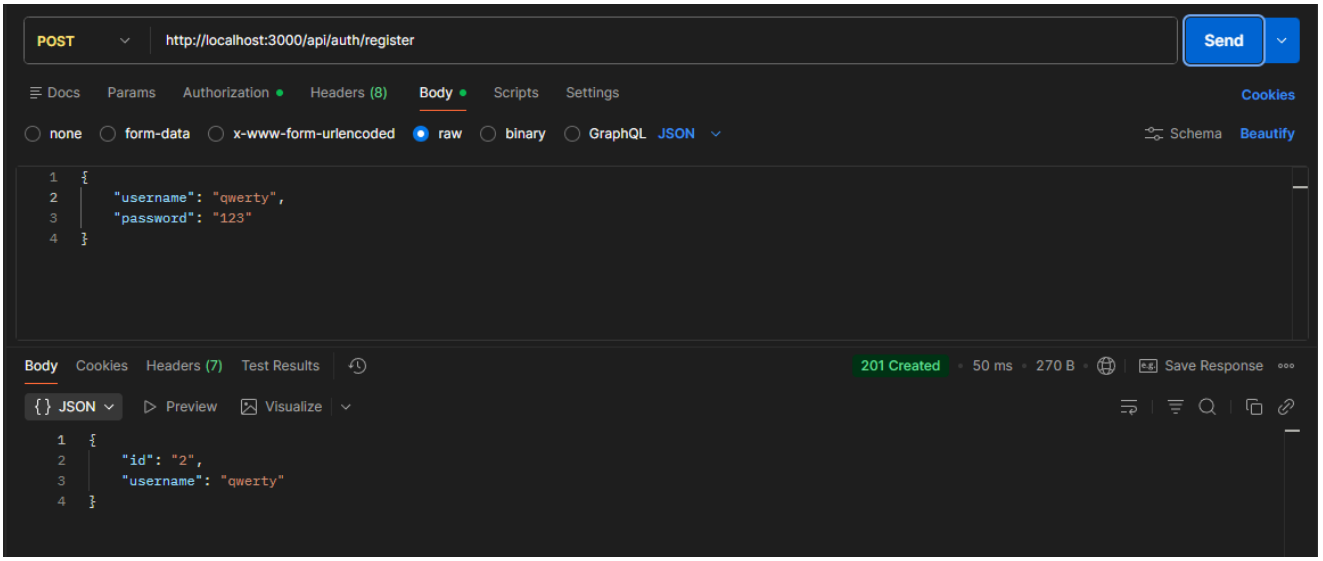
```
      error: "username and password are required",
    });
  }

  const exists = users.some((u) => u.username === username);
  if (exists) {
    return res.status(409).json({
      error: "username already exists",
    });
  }

  const passwordHash = await bcrypt.hash(password, 10);

  const user = {
    id: String(users.length + 1),
    username,
    passwordHash,
  };

  users.push(user);

  res.status(201).json({
    id: user.id,
    username: user.username,
  });
});

app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const user = users.find((u) => u.username === username);
  if (!user) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  const isValid = await bcrypt.compare(password, user.passwordHash);
  if (!isValid) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }
```

```
  const accessToken = generateAccessToken(user);
  const refreshToken = generateRefreshToken(user);

  refreshTokens.add(refreshToken);

  res.json({
    accessToken,
    refreshToken,
  });
});

app.post("/api/auth/refresh", (req, res) => {
  const { refreshToken } = req.body;

  if (!refreshToken) {
    return res.status(400).json({
      error: "refreshToken is required",
    });
  }

  if (!refreshTokens.has(refreshToken)) {
    return res.status(401).json({
      error: "Invalid refresh token",
    });
  }

  try {
    const payload = jwt.verify(refreshToken, REFRESH_SECRET);

    const user = users.find((u) => u.id === payload.sub);
    if (!user) {
      return res.status(401).json({
        error: "User not found",
      });
    }

    // Ротация refresh-токена:
    // старый удаляем, новый создаём
    refreshTokens.delete(refreshToken);

    const newAccessToken = generateAccessToken(user);
    const newRefreshToken = generateRefreshToken(user);

    refreshTokens.add(newRefreshToken);

    res.json({
      accessToken: newAccessToken,
      refreshToken: newRefreshToken,
    });
  } catch (err) {
```

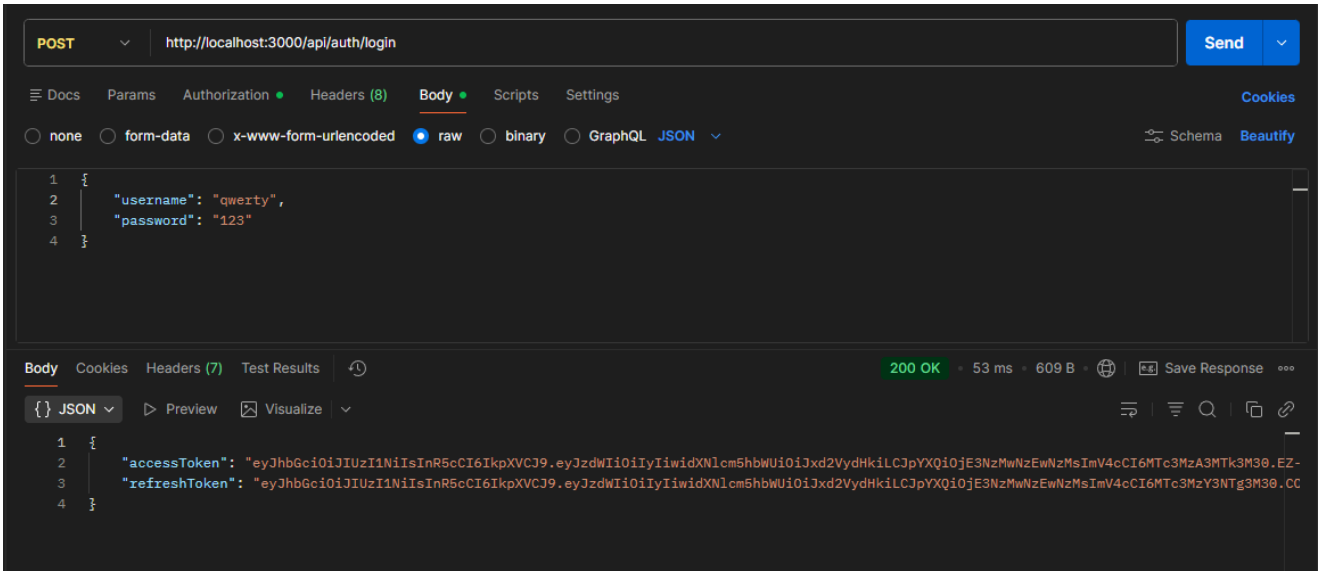
```
    return res.status(401).json({
      error: "Invalid or expired refresh token",
    });
  }
});

app.listen(PORT, () => {
  console.log(`Сервер запущен на http://localhost:${PORT}`);
});
```

Проведём несколько тестов:

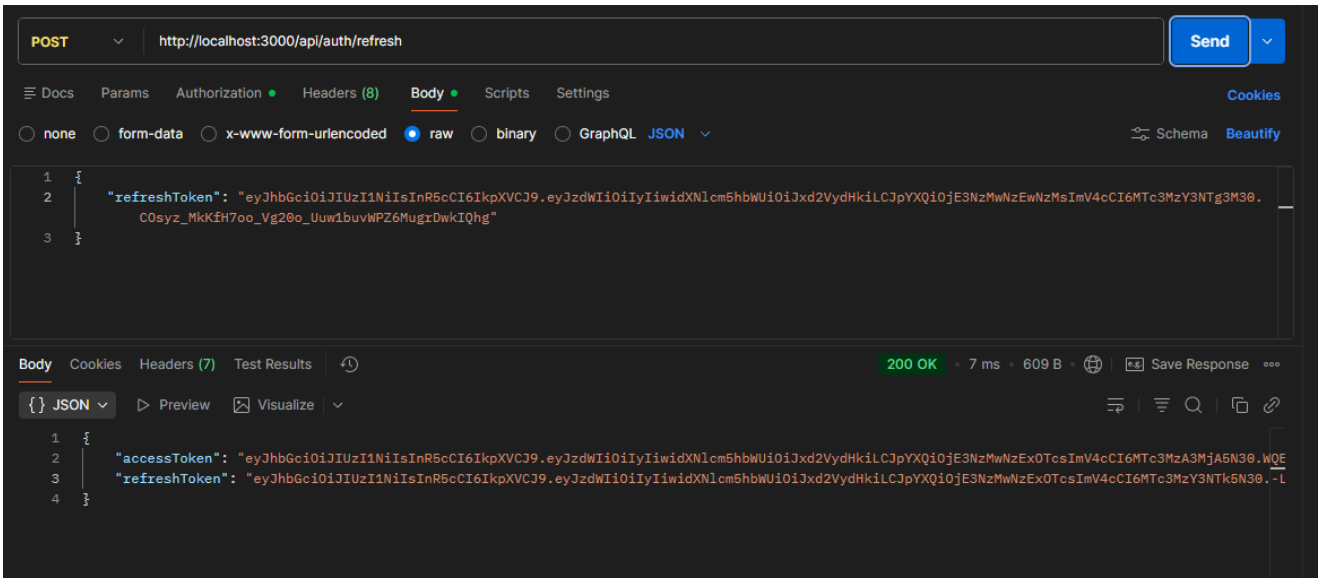
Регистрация в системе:



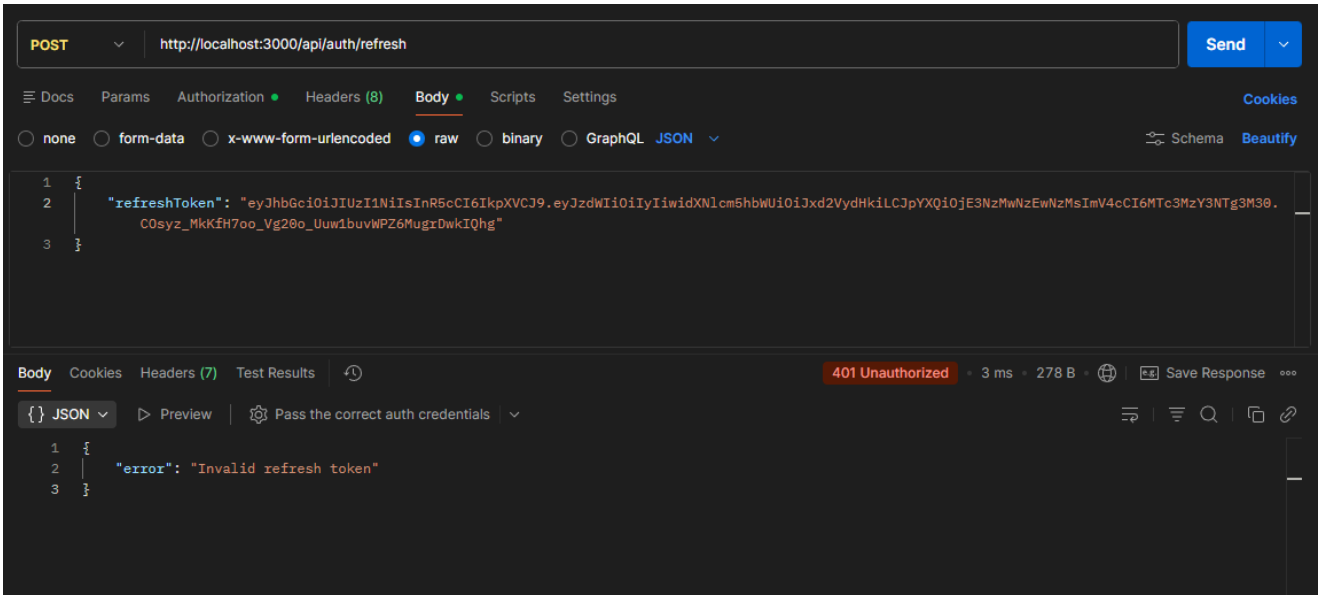
Вход в систему:



Обновление токена:



Попытка обновить токен по неактуальному referesh:



Практическое задание

Необходимо доработать программу из практического занятия №8: реализуйте генерацию refresh-токенов и добавьте маршрут:

```
POST /api/auth/refresh
```

Метод должен получать refresh-токен из заголовков и генерировать новую пару из access- и refresh-токенов.

Формат ответа от метода

```
POST /api/auth/login
```

должен соответствовать схеме:

```
{
    "accessToken": "",
    "refreshToken": ""
}
```

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой. Ссылка подгружается в соответствующий раздел СДО: Задания для самостоятельной работы -> Практические занятия 9-10.